CSI500 Stock Selection Competition - Reproducibility README

This README explains how to reproduce the submitted CSI500 portfolio pipeline end-to-end. It should accompany the final report and code submission.

1. Project goal

The task is to submit long-only, fully invested portfolios over the CSI500 universe that outperform the CSI500 index over the official evaluation windows. The official score is benchmark-relative excess return:

```
excess_return = portfolio_return - CSI500_benchmark_return
```

The submitted portfolio CSV must satisfy the official constraints:

- stock must be in `data/constituents.csv`
- at least 30 stocks with positive weight
- maximum stock weight <= 0.10
- nonnegative weights summing to 1.0 within tolerance

2. Repository layout

Place the following files in the root of the competition repository, alongside the original files from `NYUSH-ML/ml-competition-sp26`.

Original competition files

```
download_data.py
features.py
baseline_xgboost.py
validate_submission.py
score_submission.py
requirements.txt
data/
```

Added model and experiment files

```
model_v2_portfolio_aware_safe.py
sweep_portfolio_aware_checked.py
generate_and_ensemble_candidates.py
fetch_extra_data.py                 # optional supplemental financial features
```

Expected output folders

```
submissions/
reports/
reports/self_test/
```

Create them if they do not exist:

```
mkdir -p submissions reports reports/self_test
```

3. Environment setup

Python 3.10+ is recommended.

```
python -m pip install -r requirements.txt
python -m pip install lightgbm pandas numpy scipy pyarrow xgboost
```

If LightGBM installation fails on macOS because of compiler/OpenMP issues, use conda-forge:

```
conda install -c conda-forge lightgbm
```

Verify the key packages:

```
python - <<'PY'
import lightgbm, pandas, numpy, scipy, pyarrow, xgboost
print('lightgbm', lightgbm.__version__)
print('pandas', pandas.__version__)
print('numpy', numpy.__version__)
print('xgboost', xgboost.__version__)
PY
```

4. Data download and updates

First-time data download:

```
python download_data.py --start 20250101 --end 20260421
```

Before Submission 1, update through the latest available date before the Labor Day market closure.

```
python download_data.py --update --end 20260503
```

Before Submission 2, refresh again:

```
python download_data.py --update --end 20260510
```

Check the downloaded date range:

```
python - <<'PY'
import pandas as pd
prices = pd.read_parquet('data/prices.parquet')
index = pd.read_parquet('data/index.parquet')
prices['date'] = pd.to_datetime(prices['date'])
index['date'] = pd.to_datetime(index['date'])
print('prices:', prices['date'].min().date(), 'to', prices['date'].max().date())
print('index :', index['date'].min().date(), 'to', index['date'].max().date())
print(index['date'].drop_duplicates().sort_values().tail(10).dt.date.to_string(index=False
))
PY
```

The effective universe may be 499 stocks rather than 500.

5. Optional supplemental financial features

The final model can run without supplemental data. If supplemental financial indicators are available, the model will merge them automatically. In my local run, AkShare valuation and macro endpoints were unstable, so I used financial statement indicators only.

To reproduce that setup:

```
python fetch_extra_data.py --skip-valuation --skip-macro
```

This should create:

```
data/financials.parquet
```

If this file is missing, the model will still run, but the exact portfolio may differ because the feature set is smaller.

6. Phase 1 portfolio reproduction

Phase 1 used the safer portfolio-aware LightGBM configuration:

```
top_k = 30
objective = huber
weight_method = rank
filter_universe = True
```

Generate the Phase 1 portfolio:

```
python model_v2_portfolio_aware_safe.py \
  --as-of 20260430 \
  --top-k 30 \
  --objective huber \
  --weight-method rank \
  --filter-universe \
  --out submissions/phase1.csv
```

Validate it:

```
python validate_submission.py submissions/phase1.csv
```

7. Phase 2 portfolio reproduction

Phase 2 used a self-ensemble of several strong Huber LightGBM candidate configurations rather than a single aggressive portfolio. The purpose was to reduce variance and emphasize names that multiple strong configurations agreed on.

Run the ensemble generator:

```
python generate_and_ensemble_candidates.py
```

Expected final output:

```
submissions/final_self_ensemble.csv
```

Validate it:

```
python validate_submission.py submissions/final_self_ensemble.csv
```

If your local version of `generate_and_ensemble_candidates.py` supports explicit output arguments, set the output path to `submissions/final_self_ensemble.csv` to match the report.

8. Reproduce the 240-run portfolio-aware sweep

This sweep was used to choose the Phase 1 configuration and to guide the Phase 2 self-ensemble candidate family.

```
python sweep_portfolio_aware_checked.py \
  --model-script model_v2_portfolio_aware_safe.py \
  --windows \
    20260403:20260407:20260410 \
    20260410:20260414:20260417 \
    20260417:20260421:20260424 \
    20260424:20260427:20260430 \
  --top-k 30 40 50 60 80 \
  --objectives huber lambdarank \
  --weight-methods rank vol_adjusted equal \
  --filter-modes none filter \
  --no-ensemble
```

This produces:

```
reports/portfolio_sweep_checked_results.csv
reports/portfolio_sweep_checked_summary.csv
submissions/sweep/
```

The best Phase 1 family was:

```
objective = huber
weight_method = rank
top_k = 30-40
```

The selected Phase 1 submission used the filtered top-k=30 Huber/rank variant for downside control.

9. Reproduce Phase 1 historical self-test

The Phase 1 self-test used historical as-of dates and later scoring windows. Each portfolio was generated before the scored period, then evaluated against CSI500 using `score_submission.py`.

Example windows:

```
# Window 1
python model_v2_portfolio_aware_safe.py --as-of 20260403 --top-k 30 --objective huber --
weight-method rank --filter-universe --out submissions/selftest_phase1_0403.csv
python score_submission.py submissions/selftest_phase1_0403.csv --start 20260407 --end
20260410
```

```
# Window 2
python model_v2_portfolio_aware_safe.py --as-of 20260410 --top-k 30 --objective huber --
weight-method rank --filter-universe --out submissions/selftest_phase1_0410.csv
python score_submission.py submissions/selftest_phase1_0410.csv --start 20260414 --end
20260417

# Window 3
python model_v2_portfolio_aware_safe.py --as-of 20260417 --top-k 30 --objective huber --
weight-method rank --filter-universe --out submissions/selftest_phase1_0417.csv
python score_submission.py submissions/selftest_phase1_0417.csv --start 20260421 --end
20260424

# Window 4
python model_v2_portfolio_aware_safe.py --as-of 20260424 --top-k 30 --objective huber --
weight-method rank --filter-universe --out submissions/selftest_phase1_0424.csv
python score_submission.py submissions/selftest_phase1_0424.csv --start 20260427 --end
20260430
```

Reported Phase 1 self-test results:

Window	Portfolio Return	Benchmark Return	Excess Return
2026-04-07 to 2026-04-10	+7.551%	+5.766%	+1.786%
2026-04-14 to 2026-04-17	+3.655%	+3.094%	+0.562%
2026-04-21 to 2026-04-24	+1.779%	-0.508%	+2.287%
2026-04-27 to 2026-04-30	+1.424%	+1.232%	+0.192%

10. Reproduce Phase 2 historical self-test

The Phase 2 self-test evaluates the self-ensemble process. For each historical window, regenerate the ensemble at the historical cutoff, then score it only on the later unseen period.

The reported Phase 2 self-test used `generate_and_ensemble_candidates.py` followed by `score_submission.py`.

Reported Phase 2 self-test results:

Window	Portfolio Return	Benchmark Return	Excess Return
2026-04-07 to 2026-04-10	+7.545%	+5.766%	+1.779%
2026-04-14 to 2026-04-17	+3.379%	+3.094%	+0.285%
2026-04-21 to 2026-04-24	+1.461%	-0.508%	+1.970%
2026-04-27 to 2026-04-30	+1.354%	+1.232%	+0.122%

Summary:

```
Average portfolio return: +3.435%
Average benchmark return: +2.396%
Average excess return: +1.039%
Median excess return: +1.032%
Worst excess return: +0.122%
Best excess return: +1.970%
Hit rate: 4/4 = 100%
```

11. Scoring a generated submission

Use `score_submission.py` only on windows that occur after the portfolio's historical as-of date. Do not score a portfolio generated with later data on an earlier window.

Example:

```
python score_submission.py submissions/phase1.csv --start 20260506 --end 20260508
```

This only works after the relevant market data exists locally.

12. Random seeds and determinism

The LightGBM ensemble uses fixed seeds:

```
42, 43, 44, 45, 46
```

Exact portfolios may differ slightly across machines or LightGBM versions because of floating-point and parallelism differences, but the submitted portfolios should be reproducible within numerical noise when using the same data files and package versions.

13. Academic integrity

LLM assistance was used for coding, debugging, experiment planning, and writeup drafting. The final portfolios were generated by models trained from scratch on the provided/public data, not by prompting an LLM for stock picks.

14. Minimal command checklist

```
# 1. Install
python -m pip install -r requirements.txt
python -m pip install lightgbm pandas numpy scipy pyarrow xgboost

# 2. Download/update data
python download_data.py --start 20250101 --end 20260421
python download_data.py --update --end 20260503

# 3. Optional financial features
python fetch_extra_data.py --skip-valuation --skip-macro

# 4. Phase 1 portfolio
python model_v2_portfolio_aware_safe.py --as-of 20260430 --top-k 30 --objective huber --
weight-method rank --filter-universe --out submissions/phase1.csv
python validate_submission.py submissions/phase1.csv

# 5. Sweep evidence
python sweep_portfolio_aware_checked.py --model-script model_v2_portfolio_aware_safe.py --
windows 20260403:20260407:20260410 20260410:20260414:20260417 20260417:20260421:20260424
20260424:20260427:20260430 --top-k 30 40 50 60 80 --objectives huber lambdarank --weight-
methods rank vol_adjusted equal --filter-modes none filter --no-ensemble

# 6. Phase 2 final self-ensemble
python generate_and_ensemble_candidates.py
python validate_submission.py submissions/final_self_ensemble.csv
```